

SEENBIRD AI AGENT · MESSENGER SDK

AI Agent 모바일 웹뷰 연동 가이드

구성 시나리오 · 개발 영역 구분 · 런처 커스터마이징
넋지 메시지 · 푸시 알림 수신 (Android / iOS)

대상 구성: 모바일웹 / 앱 내 웹뷰 (React SDK)
기준 버전: @sendbird/ai-agent-messenger-react v1.42.0
문서 발행: 2026-10-02

목차

1. 구성 시나리오: 모바일웹 vs 앱 내 웹뷰

2. 개발 영역 구분

3. 플로팅 버튼(런처) 구현 방식

4. 안 읽은 메시지(Unread) 표시

5. 넛지 메시지(Nudge Message)

6. 런처 아이콘/스타일 변경

7. 푸시 알림 수신 (Android / iOS)

7.1 전체 구조

7.2 네이티브 SDK 필요 여부

7.3 대시보드 설정

7.4 사전 준비 (앱 측)

7.5 Android 구현

7.6 iOS 구현

7.7 웹 측 동작

7.8 유의 사항

8. 요약

1. 구성 시나리오: 모바일웹 vs 앱 내 웹뷰

먼저 두 가지 구성을 구분할 필요가 있습니다. SDK의 챗봇 UI는 두 경우 모두 동일하게 동작하지만, 진입 방식과 개발 범위가 달라집니다.

시나리오 A — 모바일웹 (모바일 브라우저에서 접속하는 웹 페이지)

귀사 웹 페이지 → SDK 임베드 → 런처(플로팅 버튼) 클릭 → 같은 페이지 내 메신저 오버레이 오픈

- 웹뷰·네이티브 개발이 필요 없습니다.
- 런처 클릭 시 열리는 것은 **같은 웹 페이지 안에서 SDK가 그려주는 메신저 UI(오버레이)**입니다. SDK가 별도의 웹뷰를 호출하는 구조가 아닙니다.

시나리오 B — 네이티브 앱 내 웹뷰

네이티브 앱 자체 버튼(귀사) → 웹뷰 오픈(귀사) → 챗봇 웹 페이지 로드(귀사 호스팅) → SDK 메신저 렌더링 (Sendbird)

- 웹뷰는 네이티브 앱이 만드는 컴포넌트이므로, 진입 버튼과 웹뷰 오픈/종료는 귀사(네이티브) 개발 영역입니다.
- 닫기 시 웹이 브릿지로 `quit` 이벤트를 보내고, 네이티브가 웹뷰를 닫는 처리를 해야 합니다. 세션 토큰 재발급 등 브릿지 상세 스펙은 제공드린 샘플의 `WEBVIEW_BRIDGE.md`를 참고해 주세요.

2. 개발 영역 구분

구분	Sendbird 제공 영역	귀사 개발 영역
챗봇 UI	대화 화면, 대화 목록, 메시지 송수신 UI 전체 (테마·컬러·문구 커스텀 옵션 포함)	—
진입 버튼(런처)	기본 런처 제공 + 위치/크기/여백 설정 커스텀 런처 연동 인터페이스 제공	자체 디자인 버튼 사용 시 버튼 UI 구현 네이티브 앱 구성 시 앱 내 진입 버튼
호스팅 페이지	—	SDK를 임베드할 웹 페이지 개발·호스팅
인증	세션 토큰 검증	사용자 생성·세션 토큰 발급 서버 (Platform API 연동), 토큰 만료 시 재 발급 처리
웹뷰 연동 (시나리오 B만)	—	웹뷰 오픈/종료, 웹·네이티브 브릿지 (<code>quit</code> , <code>session_token</code> , <code>context</code> 업데이트)
푸시 알림	푸시 발송, 토큰 등록 API	FCM/APNs 설정, 토큰 획득·전달, 알림 표시·탭 처리 (7장 참고)
대화·AI 응답	AI 에이전트 런타임, 지식베이스, 대시보드 전 체	에이전트 설정값 입력(대시보드 작업)

3. 플로팅 버튼(런처) 구현 방식

"기본 런처 제공 + 설정을 통한 제어"는 아래 두 가지 방식 모두 SDK에서 공식 지원합니다.

방식 1. 런처 아이콘을 직접 커스텀하는 방식 (완전 커스텀)

`FixedMessenger.Launcher` 에 귀사가 만든 컴포넌트를 전달하면 런처 버튼 UI 전체를 귀사 디자인으로 교체할 수 있습니다. SDK가 `opened` (현재 열림 상태), `setOpened` (열기/닫기 함수), `style` (배치 스타일), `size` 를 넘겨주므로, 버튼의 모양·아이콘·애니메이션은 자유롭게 구현하고 클릭 시 `setOpened(true)` 만 호출하면 챗봇 오버레이가 열립니다.

```

<FixedMessenger appId={...} aiAgentId={...} userSessionInfo={...} state={{ opened,
setOpened }}>
  <FixedMessenger.Launcher
    component={({ opened, setOpened, style, size }) => (
      // style을 그대로 적용하면 SDK가 계산한 위치에 배치되며, 자체 포지셔닝도 가능합니다
      <button style={style} onClick={() => setOpened(!opened)}>
        
      </button>
    )}
  />
</FixedMessenger>

```

이 방식은 챗봇 화면 밖의 별도 버튼(페이지 상단 메뉴, 배너 등)에서 `setOpened(true)` 를 호출해 챗봇을 여는 것도 동일하게 가능합니다.

방식 2. 내장 런처 + 열림 상태(opened) 변수 관리 방식

기본 제공 런처를 그대로 쓰면서, 위치·크기·여백은 `FixedMessenger.Style` 로, 열림/닫힘은 `opened` 상태 변수로 제어하는 방식입니다. 초기값을 `false` 로 두면 "챗봇 오버레이는 닫혀 있고 플로팅 버튼만 보이는 상태"가 되며, 버튼 클릭 시 오버레이가 열리고 닫기(X) 시 다시 버튼으로 복귀합니다.

```

const [opened, setOpened] = useState(false); // false = 버튼만 노출, 오버레이 닫힘

<FixedMessenger
  appId={...} aiAgentId={...} userSessionInfo={...}
  state={{ opened, setOpened }} // 열림 상태를 귀사 코드에서 변수로 관리
>
  <FixedMessenger.Style
    position={'end-bottom'} // 우측 하단 (start-bottom = 좌측 하단)
    launcherSize={56} // 버튼 크기(px)
    margin={{ end: 24, bottom: 28 }} // 가장자리 여백
  />
</FixedMessenger>

```

특정 페이지에서 버튼을 숨기는 등의 노출 여부 제어도 이 `opened` 변수와 렌더링 조건으로 귀사 코드에서 제어하시면 됩니다. 제공드린 샘플 프로젝트의 `SimpleChatPage.tsx` 에 방식 2가 동작하는 형태로 반영되어 있습니다.

4. 안 읽은 메시지(Unread) 표시

레드닷(새 메시지 있음 점 표시) — SDK 내장 지원

아래 설정으로 활성화하면 안 읽은 메시지가 있을 때 기본 런처에 붉은 점이 표시됩니다.

```
<FixedMessenger
...
  config={{ launcher: { unreadBadgeEnabled: true } }} // 기본값 false
/>
```

이 배지는 **접속(connect) 시점과 재접속 시점에 조회되어 표시되는** 방식으로, 실시간 갱신은 아니라는 점 참고 부탁드립니다.

숫자 카운트 표시 — 귀사 커스텀 필요

숫자 형태의 unread 카운트(예: ③)는 SDK 내장 기능으로 제공되지 않습니다. 다만 방식 1의 커스텀 런처와 조합하면 구현 가능합니다 — `useMessengerContext()` 의 Chat SDK 인스턴스로 안 읽은 메시지 수를 조회해 커스텀 버튼 위에 배지로 렌더링하는 방식입니다.

별도 가이드 제공

구현 방법과 바로 사용 가능한 참고 코드는 [Sendbird_AIAgent_커스텀런처_미읽음수_가이드.pdf](#) 문서로 별도 제공드립니다. 샘플 프로젝트의 `web/src/components/UnreadCountLauncher.tsx` 에도 동일 코드가 포함되어 있습니다.

5. 넛지 메시지(Nudge Message) — 런처 위 유도 문구 노출

넛지 메시지는 표시 영역과 트리거 영역이 나뉘어 있어, 아래와 같이 역할이 구분됩니다.

① 노출(렌더링/상태 관리): SDK 내장 영역

넛지 메시지가 수신되면 SDK가 **런처 위에 말풍선 형태로 자동 표시**하며, 다음 상태 관리가 모두 내장되어 있습니다 — 메신저가 열려 있는 동안 자동 숨김, 닫기(dismiss) 버튼, 등장 애니메이션, 그리고 **말풍선 클릭 시 대화 생성 → 메신저 오픈 → 지정된 첫 메시지 자동 전송**까지 SDK가 처리합니다. 귀사에서 별도 UI를 만들 필요가 없습니다.

② 트리거(언제 띄울지): 귀사 영역

넛지는 자동으로 뜨는 것이 아니라, 귀사 코드에서 원하는 시점(특정 페이지 진입, 일정 시간 체류, 장바구니 이탈 등)에 요청해야 합니다.

```
// messengerRef를 통해 호출 — 이벤트 키와 컨텍스트를 서버로 전달
messengerRef.current?.requestNudgeMessage({
  key: 'cart_abandon', // 트리거 이벤트 키
  context: { productName: '상품명' }, // 메시지 생성에 활용할 추가 컨텍스트
});
```

③ 메시지 내용 생성: Sendbird 서버/대시보드 설정 영역

전달된 이벤트 키와 컨텍스트를 기반으로 서버에서 노출할 문구(및 클릭 시 전송될 첫 사용자 메시지)를 생성해 내려줍니다. 문구 자체는 코드에 하드코딩하는 것이 아니라 에이전트 설정과 연동됩니다.

④ (선택) 말풍선 디자인 커스텀: 가능, 이 경우 UI는 귀사 영역

기본 말풍선 디자인을 바꾸고 싶으시면 런처 커스텀과 동일한 패턴으로 교체할 수 있습니다. SDK가 표시할 메시지(`nudgeMessage`), 닫기 함수(`dismissNudgeMessage`), 대화 시작 함수(`startNudgeConversation`), 배치 스타일(`style`)을 넘겨주므로 UI만 귀사 디자인으로 구현하면 됩니다.

```
<FixedMessenger.NudgeMessage
  component={{ { nudgeMessage, dismissNudgeMessage, startNudgeConversation, style } }}
=>
  nudgeMessage && (
    <div style={style} className="my-nudge-bubble">
      <span onClick={startNudgeConversation}>{nudgeMessage.message}</span>
      <button onClick={dismissNudgeMessage}>×</button>
    </div>
  )
}
```

6. 런처 아이콘/스타일 변경

플로팅 버튼(런처)에 표시되는 아이콘과 색상을 바꾸는 방법은 두 가지입니다.

방법 A. 대시보드 설정 (권장 — 코드 수정 불필요)

기본 런처의 아이콘 이미지·위젯 색상·자동 열림 여부는 코드가 아니라 **서버(대시보드) 설정**에서 내려옵니다. 대시보드에서 해당 에이전트의 **Channels** → **Messenger** 스타일 설정 항목에서 변경합니다.

- **런처 아이콘**: 기본 아이콘 또는 커스텀 이미지(이미지 URL 업로드/지정)
- **위젯 색상**: 런처 배경색을 포함한 메신저 테마 컬러
- **자동 열림(auto open)**: 페이지 진입 시 메신저를 자동으로 열지 여부

대시보드에서 변경하면 **iOS·Android·웹 어디서든 코드 수정이나 재배포 없이 즉시 반영**됩니다. 플랫폼별로 아이콘이 달라야 하는 경우가 아니라면 이 방식을 권장합니다.

방법 B. 코드로 완전 커스텀

대시보드 옵션 범위를 넘어 버튼 모양 자체(형태, 애니메이션, 배지 등)를 바꾸려면 3장의 방식 1과 동일하게 `FixedMessenger.Launcher` 로 런처 컴포넌트를 교체합니다.

```
<FixedMessenger.Launcher
  component=(({ opened, setOpened, style, size }) => (
    <button style={style} onClick={() => setOpened(!opened)}>
      
    </button>
  ))
/>
```

이 경우 아이콘 이미지는 귀사 웹 페이지의 리소스이므로 교체/버전 관리도 귀사 배포 프로세스를 따릅니다. 기본 런처와 동일한 룩앤필을 유지하고 싶다면 SDK가 export하는 `LauncherBase` 컴포넌트를 재사용하면서 `imageUrl` 만 바꾸는 방법도 있습니다.

7. 푸시 알림 수신 (Android / iOS)

사용자가 상담 화면에 진입해 있지 않은 상태에서 새 메시지를 알려려면 앱 푸시 알림이 필요합니다. 웹뷰 구성에서는 **네이티브 앱에 Sendbird SDK가 없으므로**, 토큰 등록 경로가 네이티브 SDK 구성과 다릅니다. 본 장은 제공드린 샘플에 실제로 적용·빌드 검증된 코드를 기준으로 작성되었습니다.

7.1 전체 구조

네이티브는 **토큰 획득과 알림 표시**만 담당하고, **Sendbird 등록은 웹의 SDK가** 수행합니다. 토큰은 기존 브릿지(`push_token`)를 통해 네이티브에서 웹으로 전달됩니다.

[토큰 등록]

FCM / APNs → 네이티브(토큰 수신·보관) → 웹뷰 로드 완료 시 `bridge.sendPushToken()`
 → 웹: `PushTokenRegister` → `chatSDK.registerFCM/APNSPushTokenForCurrentUser(token)`

[메시지 수신]

Sendbird 서버 → FCM / APNs → 네이티브(알림 표시) → 사용자 탭 → 상담 웹뷰 오픈

토큰 전달 타이밍

푸시 토큰은 앱 시작 직후, 즉 **웹뷰가 열리기 전에** 도착합니다. 따라서 수신 즉시 전달하면 유실되므로, 샘플은 토큰을 보관해 두었다가 **웹뷰 로드 완료 시점에 전달**하고, 이후 토큰이 갱신되면 즉시 전달하도록 콜백을 등록합니다.

7.2 네이티브에 Sendbird SDK 추가가 필요한가요?

기본적인 푸시 수신에는 **필요하지 않습니다**. 웹뷰 구성에서는 토큰 등록을 웹의 SDK가 담당하고, 네이티브는 OS의 FCM/APNs 기능만 사용하기 때문입니다. 기능별로 정리하면 다음과 같습니다.

기능	웹뷰 구성에서의 처리	네이티브 Sendbird SDK
푸시 토큰 획득	FCM / APNs 등 OS 표준 API	불필요
토큰 등록 / 해제	웹 SDK의 <code>registerFCM/APNSPushTokenForCurrentUser()</code>	불필요
알림 표시	네이티브가 로컬 알림을 직접 생성	불필요
알림 탭 → 화면 이동	네이티브 Intent / Delegate 처리	불필요
클릭 통계	웹 SDK의 <code>markPushNotificationAsClicked()</code> (탭 시 payload를 브릿지로 웹에 전달)	불필요
전달(delivered) 통계	앱이 실행되지 않은 수신 시점에 기록해야 함	필요 (선택 기능)

전달 통계가 필요한 경우에만 선택적으로 추가

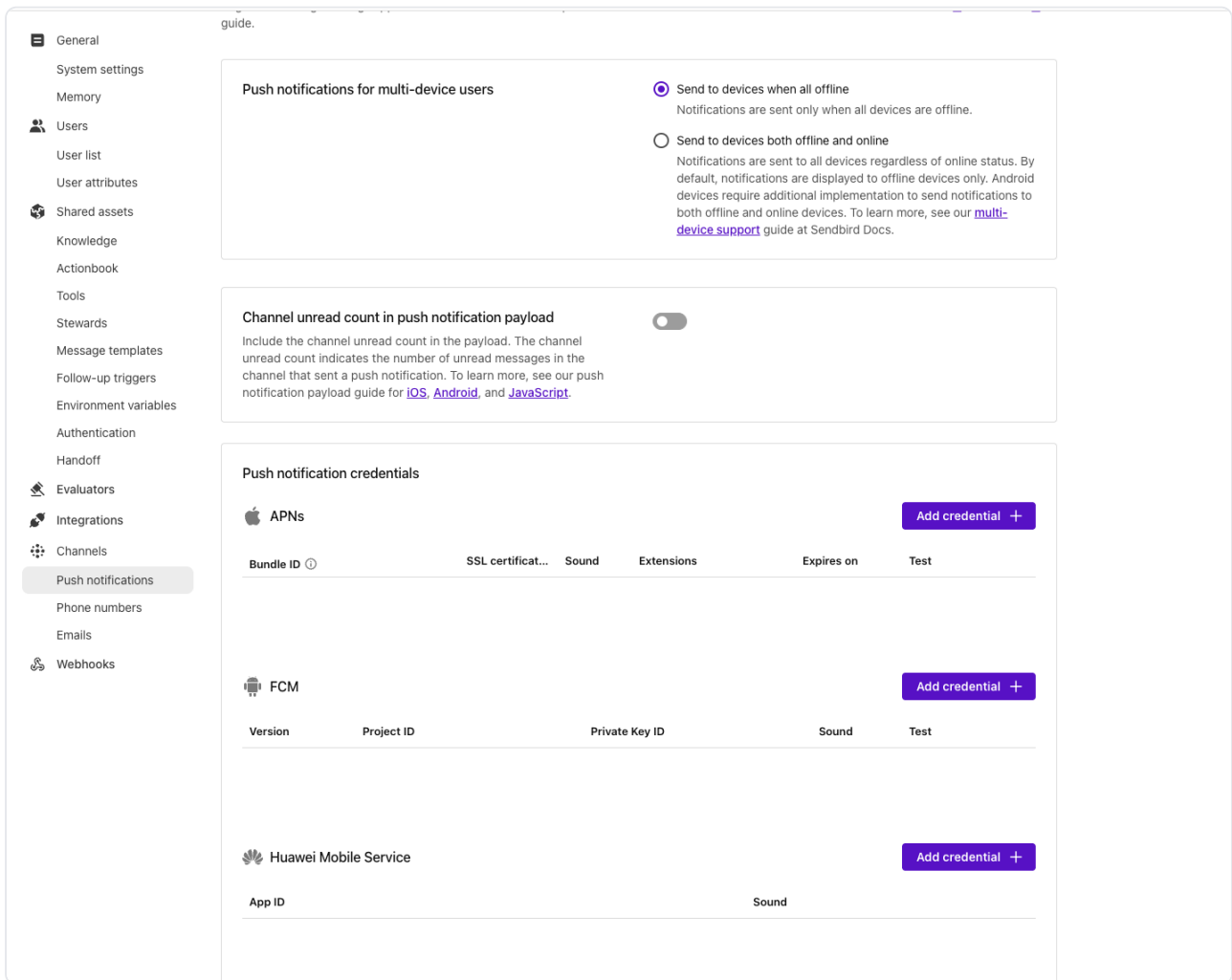
푸시 "전달률" 집계는 알림이 도착한 시점(앱 미실행)에 기록해야 하는데, 이때 웹뷰는 떠 있지 않아 웹 SDK로 호출할 수 없습니다. 이 지표가 필요하다면 iOS는 **Notification Service Extension**에, Android는 수신 서비스에 Chat SDK를 추가해 `markPushNotificationAsDelivered()`를 호출해야 합니다. 알림 수신·표시 자체에는 영향이 없으므로 대부분의 경우 생략 가능합니다.

※ Sendbird 공식 네이티브 샘플(`delight-ai-agent`)의

`AIAgentMessenger.registerPush(deviceToken:)` 방식은 네이티브 SDK로 메신저 UI까지 구현하는 구성을 전제로 합니다. 본 프로젝트처럼 웹뷰로 UI를 구성하는 경우에는 해당 API 대신 위 표의 브릿지 방식을 사용합니다.

7.3 Sendbird 대시보드 설정

푸시를 발송하려면 대시보드에 각 플랫폼의 인증 정보를 등록해야 합니다. **Settings** → **Channels** → **Push notifications** 메뉴에서 설정합니다.



1) 인증 정보 등록 (Push notification credentials)

플랫폼	등록 항목	발급 위치
APNs (iOS)	Bundle ID + SSL 인증서(.p12) + Sound(선택)	Apple Developer → Certificates, Identifiers & Profiles
FCM (Android)	Firebase 서비스 계정 키(JSON) — 등록 후 Project ID / Private Key ID 표시	Firebase 콘솔 → 프로젝트 설정 → 서비스 계정
Huawei (선택)	App ID / App Secret	Huawei Developer 콘솔 — HMS 단말 대응이 필요한 경우만

각 항목의 **Add credential** 버튼으로 등록하며, 등록 후 **Test** 버튼으로 실제 발송을 검증할 수 있습니다. 개발/운영 환경의 Bundle ID가 다르면 각각 등록해야 합니다.

APNs 인증서 만료 관리

APNs SSL 인증서는 유효기간(보통 1년)이 있으며 대시보드의 **Expires on** 열에 표시됩니다. 만료되면 iOS 푸시가 전부 중단되므로, 만료 전 갱신·재등록이 필요합니다.

2) 다중 기기 사용자 발송 정책 (Push notifications for multi-device users)

옵션	동작
Send to devices when all offline (기본)	사용자의 모든 기기가 오프라인일 때만 발송합니다.
Send to devices both offline and online	온라인 여부와 무관하게 발송합니다. (Android는 온라인 수신에 추가 구현이 필요)

웹뷰 구성에서는 기본값(all offline)을 권장합니다

웹뷰가 열려 있는 동안에만 Sendbird 연결이 유지되므로, **상당 화면을 보고 있으면 "온라인"**으로 간주되어 푸시가 발송되지 않고, **웹뷰를 닫으면 "오프라인"**이 되어 푸시가 발송됩니다. 즉 기본값만으로도 "보고 있을 때 중복 알림 없음 / 안 보고 있을 때 알림 수신"이 자연스럽게 동작합니다.

3) 안 읽은 메시지 수를 payload에 포함 (Channel unread count in push notification payload)

이 토글을 켜면 푸시 payload에 해당 채널의 안 읽은 메시지 수가 포함됩니다. 네이티브에서 **앱 아이콘 뱃지 수를 표시**하거나 알림 문구에 건수를 노출할 때 활용할 수 있습니다. (기본값 꺼짐)

7.4 사전 준비 (앱 측)

구분	준비 사항
Android	Firebase 프로젝트 생성 → <code>google-services.json</code> 을 <code>app/</code> 에 추가 → Firebase 서버 키를 Sendbird 대시보드에 등록
iOS	Xcode에서 Push Notifications Capability 추가 → Apple Developer 콘솔에서 APNs 인증서/키 발급 → Sendbird 대시보드에 등록
공통	위 7.3의 대시보드 인증 정보 등록 완료

시뮬레이터/에뮬레이터 제약

iOS 시뮬레이터는 APNs 토큰이 발급되지 않으므로 푸시 수신 테스트는 **실기기**가 필요합니다. Android 에뮬레이터는 Google Play 이미지에서 FCM 수신이 가능합니다.

7.5 Android 구현

1) Gradle 설정

```
// 루트 build.gradle.kts
plugins {
    id("com.google.gms.google-services") version "4.4.2" apply false
}

// app/build.gradle.kts
plugins {
    id("com.google.gms.google-services")
}
dependencies {
    implementation("com.google.firebase:firebase-messaging:24.0.0")
}
```

※ google-services 플러그인은 `google-services.json` 이 없으면 빌드가 실패합니다. 샘플에는 의존성만 추가되어 있으므로, Firebase 설정 파일을 넣은 뒤 플러그인을 활성화해 주세요.

2) AndroidManifest.xml

```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

<service
    android:name=".MyFirebaseMessagingService"
    android:exported="false">
    <intent-filter>
        <action android:name="com.google.firebase.MESSAGING_EVENT" />
    </intent-filter>
</service>
```

3) 푸시 수신 서비스 — `MyFirebaseMessagingService.kt`

Sendbird는 **data 메시지**로 발송하므로 OS가 알림을 자동 표시하지 않습니다. `data["sendbird"]` 를 파싱해 로컬 알림을 직접 생성해야 합니다.

```
class MyFirebaseMessagingService : FirebaseMessagingService() {

    /** FCM 토큰 발급·갱신 시 호출 — 보관해 두었다가 웹뷰 로드 후 전달 */
    override fun onNewToken(token: String) {
        PushTokenHolder.updateToken(token)
    }

    /** 푸시 수신 — Sendbird payload를 파싱해 로컬 알림 생성 */
    override fun onMessageReceived(remoteMessage: RemoteMessage) {
        val sendbirdPayload = remoteMessage.data["sendbird"] ?: return // 타 서비스 푸시는 무시
    }
}
```

```

        val sendbird = JSONObject(sendbirdPayload)
        val message = sendbird.optString("message")
        val channelUrl = sendbird.optJSONObject("channel")?.optString("channel_url")
        val senderName = sendbird.optJSONObject("sender")?.optString("name")

        showNotification(title = senderName, body = message, channelUrl =
channelUrl)
    }

    private fun showNotification(title: String, body: String, channelUrl: String?) {
        createNotificationChannelIfNeeded()

        // 알림 탭 시 상담 웹뷰로 이동 (채널 URL이 있으면 해당 대화로 바로 진입)
        val intent = Intent(this, WebViewActivity::class.java).apply {
            flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TOP
            putExtra(WebViewActivity.EXTRA_ROUTE, "simple")
            channelUrl?.let { putExtra(WebViewActivity.EXTRA_INITIAL_CHANNEL_URL,
it) }
        }

        val pendingIntent = PendingIntent.getActivity(
            this, 0, intent,
            PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE,
        )

        val notification = NotificationCompat.Builder(this, CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_notification) // 자사 아이콘으로 교체
            .setContentTitle(title)
            .setContentText(body)
            .setAutoCancel(true)
            .setPriority(NotificationCompat.PRIORITY_HIGH)
            .setContentIntent(pendingIntent)
            .build()

        NotificationManagerCompat.from(this).notify(NOTIFICATION_ID, notification)
    }
}

/** 토큰 보관소 — 웹뷰가 열리기 전에 도착한 토큰을 유실 없이 전달하기 위함 */
object PushTokenHolder {
    @Volatile var token: String? = null
    private set

    @Volatile var onTokenReady: ((String) -> Unit)? = null

    fun updateToken(newToken: String) {
        token = newToken
        onTokenReady?.invoke(newToken)
    }
}

```

4) 웹뷰 로드 완료 시 토큰 전달 — `WebViewActivity.kt`

```
webViewClient = object : WebViewClient() {
    override fun onPageFinished(view: WebView?, url: String?) {
        super.onPageFinished(view, url)

        // 이미 받아둔 토큰이 있으면 즉시 전달, 아직이면 수신 시점에 전달되도록 등록
        PushTokenHolder.token?.let { bridge.sendPushToken(it) }
        PushTokenHolder.onTokenReady = { token ->
            runOnUiThread { bridge.sendPushToken(token) }
        }
    }
}
```

5) 알림 권한 요청 (Android 13+) — `MainActivity.kt`

```
private val notificationPermissionLauncher =
    registerForActivityResult(ActivityResultContracts.RequestPermission())
{ isGranted ->
    Log.d("SendbirdPush", "알림 권한 결과: $isGranted")
}

private fun requestNotificationPermissionIfNeeded() {
    if (Build.VERSION.SDK_INT < Build.VERSION_CODES.TIRAMISU) return

    val granted = ContextCompat.checkSelfPermission(
        this, Manifest.permission.POST_NOTIFICATIONS,
    ) == PackageManager.PERMISSION_GRANTED

    if (!granted)
        notificationPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS)
}
```

7.6 iOS 구현

1) Capability 추가

Xcode에서 타겟 선택 → **Signing & Capabilities** → **+ Capability** → **Push Notifications**를 추가합니다. 이 설정이 없으면 APNs 토큰이 발급되지 않습니다.

2) 푸시 권한·토큰 처리 — `PushManager.swift`

SwiftUI 앱에는 AppDelegate가 없으므로, `UIApplicationDelegateAdaptor`로 연결합니다.

```
/// APNs 토큰을 보관하고 웹뷰에 전달 (웹뷰가 열리기 전 도착하는 토큰 유실 방지)
final class PushManager {
    static let shared = PushManager()
    private(set) var deviceToken: String?
    var pendingNotificationPayload: [AnyHashable: Any]?
}
```

```

var onTokenReady: ((String) -> Void)?

func updateDeviceToken(_ token: String) {
    self.deviceToken = token
    onTokenReady?(token)
}

/// Sendbird가 보낸 푸시인지 확인 (타 서비스 푸시와 구분)
static func isSendbirdPush(_ userInfo: [AnyHashable: Any]) -> Bool {
    return userInfo["sendbird"] != nil
}

class AppDelegate: NSObject, UIApplicationDelegate {
    func application(
        _ application: UIApplication,
        didFinishLaunchingWithOptions launchOptions:
[UIApplication.LaunchOptionsKey: Any]? = nil
    ) -> Bool {
        registerForPushNotifications()
        return true
    }

    private func registerForPushNotifications() {
        let center = UNUserNotificationCenter.current()
        center.delegate = self
        center.requestAuthorization(options: [.alert, .badge, .sound]) { granted,
error in
            guard granted, error == nil else { return }
            DispatchQueue.main.async { // APNs 등록은 메인 스레드에
서
                UIApplication.shared.registerForRemoteNotifications()
            }
        }
    }

    /// APNs 토큰 수신 — Data를 hex 문자열로 변환해 보관 (웹 SDK가 요구하는 형식)
    func application(
        _ application: UIApplication,
        didRegisterForRemoteNotificationsWithDeviceToken deviceToken: Data
    ){
        let token = deviceToken.map { String(format: "%02x", $0) }.joined()
        PushManager.shared.updateDeviceToken(token)
    }
}

extension AppDelegate: UNUserNotificationCenterDelegate {
    /// 포그라운드 수신 시 표시 방식 (상단 화면을 보는 중에는 빈 배열로 숨길 수 있음)
    func userNotificationCenter(
        _ center: UNUserNotificationCenter,
        willPresent notification: UNNotification,
        withCompletionHandler completionHandler: @escaping
(UNNotificationPresentationOptions) -> Void
    ){

```

```

        completionHandler([.banner, .sound, .badge])
    }

    /// 알림 탭 — Sendbird 푸시인 경우 상담 화면으로 이동
    func userNotificationCenter(
        _ center: UNUserNotificationCenter,
        didReceive response: UNNotificationResponse,
        withCompletionHandler completionHandler: @escaping () -> Void
    ){
        let userInfo = response.notification.request.content.userInfo
        guard PushManager.isSendbirdPush(userInfo) else {
            completionHandler(); return
        }

        PushManager.shared.pendingNotificationPayload = userInfo
        NotificationCenter.default.post(name: .didTapSendbirdPush, object: nil)
        completionHandler()
    }
}

```

3) AppDelegate 연결 — `WebViewAppApp.swift`

```

@main
struct WebViewAppApp: App {
    @UIApplicationDelegateAdaptor(AppDelegate.self) var appDelegate

    var body: some Scene {
        WindowGroup { ContentView() }
    }
}

```

4) 웹뷰 로드 완료 시 토큰 전달 — `WebViewScreen.swift`

```

func webView(_ webView: WKWebView, didFinish navigation: WKNavigation!) {
    // 이미 받아둔 토큰이 있으면 즉시 전달, 아직이면 수신 시점에 전달되도록 등록
    if let token = PushManager.shared.deviceToken {
        sendPushToken(token: token)
    }

    PushManager.shared.onTokenReady = { [weak self] token in
        DispatchQueue.main.async { self?.sendPushToken(token: token) }
    }
}

```

7.7 웹 측 동작 (수정 불필요)

웹은 이미 구현되어 있습니다. 브릿지로 토큰을 받아 플랫폼에 맞는 등록 API를 호출합니다 (`web/src/components/PushTokenRegister.tsx`).


```
bridge.on('push_token', (data: { token: string }) => {
  const platform = bridge.getPlatform();
  if (!data.token || !sdkUser) return; // 로그인(연결) 완료 후에만 등록 가능

  if (platform === 'ios') {
    chatSDK.registerAPNSPushTokenForCurrentUser(data.token);
  } else if (platform === 'android') {
    chatSDK.registerFCMPushTokenForCurrentUser(data.token);
  }
});
```

7.8 유의 사항

1) 등록은 사용자 인증 이후에만 가능합니다

푸시 토큰은 "현재 로그인한 사용자"에 귀속되므로, 웹의 SDK 연결(`sdkUser` 존재)이 완료된 뒤에 등록해야 합니다. 샘플은 이 조건을 확인한 뒤 등록합니다.

2) 로그아웃 시 토큰 해제

사용자 로그아웃이나 계정 전환 시에는 토큰을 해제해야 다른 사용자에게 알림이 가지 않습니다. 웹에서 `chatSDK.unregisterFCMPushTokenForCurrentUser(token)` 계열 API를 호출하세요.

3) 타 서비스 푸시와의 구분

앱이 다른 용도의 푸시도 사용한다면, `data["sendbird"]` (Android) / `userInfo["sendbird"]` (iOS) 존재 여부로 분기해 기존 로직과 충돌하지 않도록 해야 합니다. 샘플 코드가 이 분기를 포함하고 있습니다.

4) 비회원(익명) 사용자

익명 사용자는 접속마다 새로 발급될 수 있어 푸시 대상이 안정적으로 유지되지 않습니다. 푸시가 필요한 시나리오라면 회원 인증 기반 사용을 권장합니다.

5) 포그라운드 중복 알림

상당 화면을 보고 있는 중에도 배너가 뜨면 사용자 경험이 저하됩니다. iOS는 `willPresent` 에서 빈 배열을 반환하고, Android는 웹뷰 활성 여부를 확인해 알림 표시를 생략하는 처리를 권장합니다.

8. 요약

요구사항	지원 여부	구현 주체
버튼 디자인/아이콘 완전 커스텀	✅ <code>FixedMessenger.Launcher</code>	버튼 UI는 귀사, 연동 인터페이스는 SDK 제공
내장 버튼의 위치/크기/여백 조정	✅ <code>FixedMessenger.Style</code>	SDK 설정
런처 아이콘 이미지/위젯 색상 변경	✅ 대시보드 Channels → Messenger 설정	대시보드 설정 (코드 수정·재배포 불필요)
버튼만 보이고 챗봇 닫힌 초기 상태	✅ <code>state={{ opened, setOpened }}</code>	상태 변수는 귀사 관리
새 메시지 레드닷	✅ <code>config.launcher.unreadBadgeEnabled</code> (실시간 아님)	SDK 설정
숫자 unread 카운트	⚠️ 내장 미제공 (별도 가이드 제공)	귀사 커스텀 (SDK 조회 API 활용)
넛지 메시지 노출	✅ SDK 내장 (기본 말풍선 UI 포함)	트리거 호출·조건은 귀사, 문구는 대시보드 설정
푸시 알림 수신	✅ 지원 (7장)	FCM/APNs 설정·알림 표시는 귀사, 토큰 등록은 웹 SDK가 수행

※ 본 가이드는 Sendbird 내부 검증 결과를 토대로 작성되었습니다. 사양 변경이 있을 수 있으므로 운영 적용 전 담당 CSM 또는 SE에게 최신 상태를 확인해 주시기 바랍니다.